Spam Classifier — Project Documentation

Oct 12, 2026 · @pixel

Overview

This is a Gmail-style webmail prototype with a spam classifier built entirely from real, publicly released email data and a hand-written Multinomial Naive Bayes algorithm (no scikit-learn, no synthetic training examples). The live app has four real mail folders (Inbox, Starred, Sent, Spam), a separate Spam Detector tab for testing arbitrary text, and an About tab that reports real, freshly-computed accuracy numbers rather than hardcoded ones.

The project went through two builds. The first attempt (by another tool, before this conversation) trained on 50 hand-written example emails and misclassified real spam as "not spam." Everything in this document describes the second, from-scratch rebuild: real data (33,716 emails from the Enron-Spam corpus), a held-out evaluation methodology, and a UI built to withstand actual use rather than just a demo click-through.

Requirements (as given)

The final build brief, exactly as requested:

1. Rewrite the existing (broken) classifier from scratch into a new, advanced prototype that accurately classifies spam "without any risks."
2. No synthetic dataset and no inaccurate model — real training data only.
3. Add a separate **Spam Detector** tab; composing a new email must **not** auto-check for spam anymore — checking happens only in that tab.
4. 30 randomly-selected emails in the inbox.
5. 5 preset example emails in the Spam Detector tab.
6. Classification must work on context/meaning ("as a sentence"), not just keyword matching.
7. Must use Naive Bayes, and must show **how** Naive Bayes is being applied — explainability, not a black box.
8. "No mistakes."

A follow-up round added: working search/star/menu controls, real Starred/Sent/Spam folders (not just a flat inbox), a live confusion matrix on the About tab, and an agree/disagree feedback mechanism on Spam Detector results.

Development journey

1. **Initial plan.** First pass recommended the Enron-Spam and SpamAssassin corpora as approved, real-data sources, with a build prompt for a Gmail clone with Naive Bayes spam detection.
2. **Dataset complaint.** Those sources were reported as "not working and too old." The plan was revised to recommend Hugging Face's `ealvaradob/phishing-dataset` as primary and Kaggle's `naserabdullahalam/phishing-email-dataset` as a fallback.
3. **Diagnosing the broken first build.** A different tool had already built an app from that plan. It was trained on only **50 hand-written example emails** — exactly the "fake data" the brief had ruled out — and misclassified real spam as ham. Root cause confirmed by reproducing the failure and by retraining the *same, unmodified* classifier code on the full real corpus: a job-scam email that scored 0.01% spam probability under the 50-example dataset scored 99.62% spam under the real one, with zero changes to the algorithm.
4. **Full rebuild ordered.** Rewrite everything, use only real data, add a separate Spam Detector tab, 30 random inbox emails, 5 presets, context-aware (not keyword-only) classification, and visible Naive Bayes reasoning.
5. **Sourcing real data from a restricted sandbox.** Hugging Face, Kaggle, and the original academic corpus host were all unreachable from the build environment (blocked by its network allowlist). `raw.githubusercontent.com` was reachable, so the real Enron-Spam corpus was downloaded from a public GitHub mirror of that exact academic dataset instead.
6. **Built the classifier, dataset split, and evaluation.** A from-scratch Multinomial Naive Bayes classifier, a held-out demo pool (35 real emails never trained on), and an honest 80/20 accuracy evaluation.
7. **Built the Flask app and Gmail-style frontend**, delivered first as a downloadable zip (the build environment couldn't reach the user's computer yet), then placed directly into the requested folder once the desktop app connected and folder access was granted.

8. **Readability fix.** The downloaded corpus text turned out to be pre-tokenized by whoever built the mirror (`"don ' t be a fuddy - duddy"`, `"confirm : $ 30714"`) — technically correct but unreadable. Wrote a display-only detokenizer (`textclean.py`) that doesn't touch training data or classification.
9. **Feature round.** In response to feedback that the app "didn't look like Gmail" and several controls were inert: added real Starred/Sent/Spam folders, wired up search/star/menu, added a live confusion matrix to the About tab, and added an agree/disagree feedback control on Spam Detector results (which is honest that this batch-trained model can't retrain itself live).

Dataset

Source: the Enron-Spam corpus — V. Metsis, I. Androutsopoulos, G. Paliouras, "Spam Filtering with Naive Bayes — Which Naive Bayes?", CEAS 2006. 17,171 real spam emails and 16,545 real legitimate emails actually sent to/from Enron employees, 33,716 total. Downloaded via a public GitHub mirror, MWiechmann/enron_spam_data, which packages the same per-message files into one CSV. No row in the project's data files is hand-written, edited, or AI-generated — every one is a slice of the real downloaded corpus.

Held-out methodology. Before training, 35 real emails were set aside permanently (`data/demo_pool.csv`) — these, and only these, populate the inbox (30 randomly sampled each server start) and the Spam Detector's 5 presets, so every email shown in the running app is genuinely out-of-sample. The remaining 33,681 emails (`data/spam_dataset.csv`) are the training set.

Presentability filter. The demo pool (the 35 emails people actually see) was filtered to exclude graphic/explicit spam content common in this 2001-era corpus — matched after normalizing for the deliberate misspellings and spacing tricks that real spam uses to dodge filters (e.g. `v1agra`, `cia1is`, `x a n a x`). This filter touches **only** which of the 35 display emails were picked; the training set itself is untouched and unfiltered.

	Count
Total real emails in corpus	33,716
Spam / Ham in corpus	17,171 / 16,545
Held out for inbox + presets (never trained on)	35
Training set	33,681

Earlier sources considered and dropped: the SpamAssassin public corpus (part of the first plan) and, later, Hugging Face's `ealvaradob/phishing-dataset` and Kaggle's `naserabdullahalam/phishing-email-dataset` (recommended after the first sources were reported as outdated). All three were abandoned once building started — not because they were bad sources, but because Hugging Face, Kaggle, and the original academic corpus host were all unreachable from the build sandbox's restricted network.

Methodology — Multinomial Naive Bayes

Written from scratch in Python (no scikit-learn), so every step is inspectable and explainable in the UI. For an email, Bayes' theorem gives the probability it's spam given its words:

```
P(\text{Spam} \mid \text{words}) = \frac{P(\text{words} \mid \text{Spam}) \, P(\text{Spam})}{P(\text{words})}
```

`P(words)` is identical for both classes, so only `P(words|Spam)P(Spam)` vs. `P(words|Ham)P(Ham)` need comparing. The "naive" assumption — that each word's presence is independent of the others given the class — turns `P(words|Spam)` into a simple product over every word:

```
P(\text{words} \mid \text{Spam}) = \prod_{i=1}^{n} P(w_i \mid \text{Spam})
```

Each `P(w|Spam)` is learned from training counts with Laplace (additive) smoothing, `α = 1.0`, so a word never seen in one class doesn't force that class's probability to exactly zero the first time it appears:

```
P(w \mid \text{Spam}) = \frac{\text{count}(w, \text{Spam}) + \alpha}{\text{total words in Spam} + \alpha \times |\text{vocabulary}|}
```

Multiplying thousands of small probabilities underflows floating point, so the implementation sums their logs instead (equivalent math), then converts the final log-odds back to a 0–100% probability with a sigmoid. Priors `P(Spam)` and `P(Ham)` are simply each class's share of the training data.

Because the decision is a *statistical combination of every word in the email*, not a fixed blocklist, it generalizes to spam phrased in new ways — satisfying the "context, not just keyword matching" requirement — while still being fundamentally a word-frequency model rather than a deep sequence model.

Implementation

File	Purpose
`prepare_dataset.py`	One-time script: downloads the real Enron-Spam corpus, builds the training CSV and the held-out demo-pool CSV, applies the presentability filter to the demo pool only
`classifier.py`	`BayesianSpamClassifier` — the Naive Bayes implementation itself: tokenizer, training, log-space classification, per-word spamminess scoring, and spam-word highlighting (`<mark>` tags) for the UI
`textclean.py`	Display-only detokenizer that fixes the corpus's pre-tokenized punctuation spacing and lowercase text so emails read naturally; never touches training data or classification
`dataset.py`	Loads the training CSV; builds the 30-email random inbox sample and the 5 fixed Spam Detector presets from the held-out pool, with synthesized (display-only) sender names since the raw corpus has no sender field
`evaluate.py`	Runs an honest stratified 80/20 train/test split and reports accuracy, precision, recall, F1, and the confusion matrix; exposes `evaluate_holdout()`, reused live by `app.py`

File	Purpose
`app.py`	Flask backend — all routes (inbox, email detail, star/read toggles, compose/send, sent folder, Spam Detector check + presets + feedback, live stats)
`templates/index.html`	Single-page app shell
`static/css/style.css`	Gmail-style visual design
`static/js/app.js`	All frontend logic: folder rendering, search, star/read toggling, compose, Spam Detector explanation rendering, About-tab confusion matrix

The classifier trains fresh from the CSV every time `app.py` starts (a few seconds) — nothing is pickled or cached between runs, so the live model always matches whatever is in `data/spam_dataset.csv`.

Features

Feature	Detail
Inbox	30 real emails, randomly sampled on every server start from the 35-email held-out pool; spam rows tagged and highlighted in red
Starred	Click a star to add/remove an email; persists on the server for the session
Sent	Compose → Send actually lands here (not just a toast); compose never auto-checks for spam
Spam	The inbox filtered to just what the model flagged — same model, same highlighting, a dedicated folder
Search	Filters whatever folder is open by sender/subject/snippet
Spam Detector tab	Free-text box + 5 one-click preset real emails; classification is fully separate from composing

Feature	Detail
Explainability	Every check shows priors, per-word log-probabilities, the sigmoid conversion, and a table of the specific words that pushed the result toward spam or ham, with counts and likelihood ratios
Feedback / override	Agree/disagree buttons on every Spam Detector result; a disagreement is recorded and explained honestly (no live retraining — see Limitations) rather than faking an instant fix
About this model	Dataset provenance, training size, vocabulary size, and a confusion matrix + accuracy/precision/recall/F1, computed fresh at every server start
Inline spam highlighting	Strong spam-indicator words are wrapped in `<mark>` tags with a tooltip showing their learned P(Spam|word) and how often they appeared in spam during training

Results

From a stratified 80/20 split (seed fixed for reproducibility) of the real training set — 26,944 emails trained on, 6,737 tested on, never seen during training. Computed fresh by `evaluate_holdout()` every time the server starts, not hardcoded:

Metric	Value
Accuracy	98.65%
Spam precision	98.63%
Spam recall	98.72%
F1 score	98.68%
Vocabulary size	154,643 words

Confusion matrix (6,737 held-out test emails):

	Predicted Spam	Predicted Ham
Actual Spam	3,391 (true positive)	44 (false negative)
Actual Ham	47 (false positive)	3,255 (true negative)

All 35 emails in the held-out demo pool (the ones actually shown in the inbox and Spam Detector presets) also classify correctly — so what the running app displays will always match the real corpus's own labels for those specific emails.

Libraries and tools

Name	Used for
Python 3	The entire backend and data pipeline
Flask	Web framework — routes, JSON APIs, HTML templating (`app.py`)
pandas	Reading/wrangling the downloaded corpus CSV, building the train/demo-pool split (`prepare_dataset.py`)
requests	Downloading the dataset zip over HTTP (`prepare_dataset.py`)
Python standard library	`csv`, `io`, `os`, `random`, `re`, `sys`, `zipfile`, `unicodedata`, `math`, `collections.Counter`, `typing` — the Naive Bayes math, tokenization, text cleanup, and CSV I/O all run on these, no ML library involved
HTML5 / CSS3 / vanilla JavaScript	The entire frontend — no framework (no React/Vue), no build step, no npm dependencies
Node.js	Used only once during development to syntax-check `app.js` before shipping — not a runtime dependency of the app itself

The classifier, tokenizer, and text cleanup are all hand-written Python — no `scikit-learn`, `nltk`, or other ML/NLP library is used anywhere, so every probability the app displays can be traced back to the formulas in the Methodology section above.

Resources and citations

- Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). "Spam Filtering with Naive Bayes — Which Naive Bayes?" CEAS 2006. The academic paper and original host of the Enron-Spam corpus actually used to train this project's model.
- MWiechmann/enron_spam_data — the GitHub mirror this project actually downloaded the corpus from, as a single pre-built CSV.
- Flask documentation
- pandas documentation
- requests documentation

Considered, not used in the final build:

- SpamAssassin public corpus — part of the first plan, dropped after it was reported outdated.
- Hugging Face — ealvaradob/phishing-dataset — recommended as the primary source after the first pivot; unreachable from the build sandbox when it came time to actually build.
- Kaggle — naserabdullahalam/phishing-email-dataset — recommended as a fallback; also unreachable from the build sandbox.

Limitations (stated honestly)

- **Not an online learner.** The model trains once, in a batch, when the server starts. Disagreeing with a verdict in the Spam Detector tab is recorded as feedback but does not retrain the model on the spot — a real fix means adding the corrected example to `data/spam_dataset.csv` and restarting the server.
- **Bag-of-words, not deep context.** It reasons about which words co-occur and how strongly each is statistically associated with spam — not sentence structure or meaning the way a transformer model would. Unusual phrasing built entirely from words that look equally at home in spam or ham can still fool it; the 91 misclassifications out of 6,737 held-out test emails (see Results) are exactly this kind of case.

- **Sender/inbox metadata is cosmetic.** The raw corpus has no sender field, so display names/addresses in the inbox are synthesized for the UI and are never seen by the classifier — only each email's real subject/body text is classified.
- **State resets on restart.** Starred emails, sent mail, and feedback counts live in server memory only; they're lost when `app.py` restarts. There's no database or user accounts — this is a single-user local prototype.
- **The presentability filter on the demo pool is a judgment call**, not a requirement that was asked for — made to keep a 2001-era spam corpus's more graphic content out of a prototype likely to be demoed. It affects only which 35 of 33,716 emails are shown, never the training data itself.

Setup

```
pip install -r requirements.txt

# One-time: downloads the real dataset and builds data/spam_dataset.csv +
# data/demo_pool.csv. Needs normal internet access.
python prepare_dataset.py

# Optional: print held-out accuracy/precision/recall/F1 on an 80/20 split.
python evaluate.py

# Run the app
python app.py
```

Then open `http://localhost:5000`. The app trains the classifier on startup (a few seconds); nothing is cached between runs, so it always reflects the current `data/spam_dataset.csv`.

Project structure

```
spam_claude/
├─ app.py                  Flask routes
├─ classifier.py           BayesianSpamClassifier (Naive Bayes from scratch)
├─ dataset.py              Loads training data; builds inbox sample + presets
├─ textclean.py            Display-only detokenizer for real corpus text
├─ prepare_dataset.py      One-time real-data download + train/demo split
├─ evaluate.py             Honest held-out accuracy check + reusable metrics
├─ requirements.txt        Flask, pandas, requests
├─ data/
|  ├─ spam_dataset.csv    Training set (generated by prepare_dataset.py)
|  └─ demo_pool.csv       Held-out demo set (generated by prepare_dataset.py)
├─ templates/
|  └─ index.html          Single-page app shell
└─ static/
   ├─ css/style.css       Gmail-style styling
   └─ js/app.js           Frontend logic
```

Delivered both as a downloadable zip and, once the desktop app was linked and folder access granted, written directly to `C:\Users\Vanaparthi\.gemini\antigravity\scratch\spam_claude` on the user's machine.